

Multi-Stock Financial Data Analysis and Apple Stock Price Prediction

Project Overview:

This project provides a comprehensive analysis of financial stock market data, focusing on exploratory data analysis (EDA), risk assessment, and price forecasting. While the initial analysis covers a diversified portfolio including Apple (AAPL), Coca-Cola (KO), Tesla (TSLA), and Booking Holdings (BKNG), the core of the project focuses on predicting the future closing price of Apple Inc.

Objectives:

1. Exploratory Data Analysis: Visualizing price changes, moving averages, and daily returns for multiple tech and consumer stocks.
2. Risk Analysis: Determining the correlation between stocks and calculating Value at Risk (VaR) using Monte Carlo simulations.
3. Time Series Forecasting: Comparing two distinct methodologies for stock price prediction:

* LSTM (Long Short-Term Memory): A Deep Learning recurrent neural network designed to capture long-term dependencies in sequential data.

* ARIMA (AutoRegressive Integrated Moving Average): A classical statistical model used for analyzing and forecasting time-series data based on stationarity and autocorrelations.

By comparing a state-of-the-art Deep Learning approach with a robust statistical model, this project aims to evaluate which method offers better precision in the highly volatile financial market.

```
In [104]: !pip install -q yfinance

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import yfinance as yf
from datetime import datetime

sns.set_style("whitegrid")
plt.style.use("fivethirtyeight")
```

In
[105]:

```
stocks = ["AAPL", "KO", "TSLA", "BKNG"]

end = datetime.now()
start = datetime(end.year - 1, end.month, end.day)

# Download data
data = yf.download(stocks, start=start, end=end, auto_adjust=False)

# View data
data.head()
```

[*****100%*****] 4 of 4
completed

Out[105]:

Price	Adj Close				Close	
Ticker	AAPL	BKNG	KO	TSLA	AAPL	
Date						
2025-03-17	213.071350	4528.681152	68.144585	238.009995	214.000000	
2025-03-18	211.767014	4437.499512	67.425430	225.309998	212.690002	
2025-03-19	214.305954	4556.963867	67.328247	235.860001	215.240005	
2025-03-20	213.170914	4554.125977	67.746132	236.259995	214.100006	
2025-03-21	217.322800	4582.141113	66.735428	248.710007	218.270004	

5 rows × 24 columns



Descriptive Statistics about the Data

In
[106]:

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 250 entries, 2025-03-17 to 2026-03-13
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   (Adj Close, AAPL)      250 non-null   float64
1   (Adj Close, BKNG)      250 non-null   float64
2   (Adj Close, KO)        250 non-null   float64
3   (Adj Close, TSLA)      250 non-null   float64
4   (Close, AAPL)          250 non-null   float64
5   (Close, BKNG)          250 non-null   float64
6   (Close, KO)            250 non-null   float64
7   (Close, TSLA)          250 non-null   float64
8   (High, AAPL)           250 non-null   float64
9   (High, BKNG)           250 non-null   float64
10  (High, KO)             250 non-null   float64
11  (High, TSLA)           250 non-null   float64
12  (Low, AAPL)            250 non-null   float64
13  (Low, BKNG)            250 non-null   float64
14  (Low, KO)              250 non-null   float64
15  (Low, TSLA)            250 non-null   float64
16  (Open, AAPL)           250 non-null   float64
17  (Open, BKNG)           250 non-null   float64
18  (Open, KO)             250 non-null   float64
19  (Open, TSLA)           250 non-null   float64
20  (Volume, AAPL)         250 non-null   int64
21  (Volume, BKNG)         250 non-null   int64
22  (Volume, KO)           250 non-null   int64
23  (Volume, TSLA)         250 non-null   int64
dtypes: float64(20), int64(4)
memory usage: 48.8 KB
```

In # Summary Stats
[107]: data.describe()

Out[107]:

Price	Adj Close				Close
Ticker	AAPL	BKNG	KO	TSLA	AAPL
count	250.000000	250.000000	250.000000	250.000000	250.000000
mean	237.233567	5112.256681	69.796223	370.591880	237.710320
std	29.299344	451.710290	3.483842	69.523976	29.094907
min	171.671783	3862.019775	64.766624	221.860001	172.419998
25%	209.417732	4853.314087	67.601410	319.184990	210.145000
50%	237.856644	5227.138184	69.263653	391.145004	238.309998
75%	264.307503	5448.762451	70.237816	432.584991	264.307503
max	285.922455	5781.647949	81.003021	489.880005	286.190002

8 rows × 24 columns



Exploratory Data Analysis

Closing Price

```

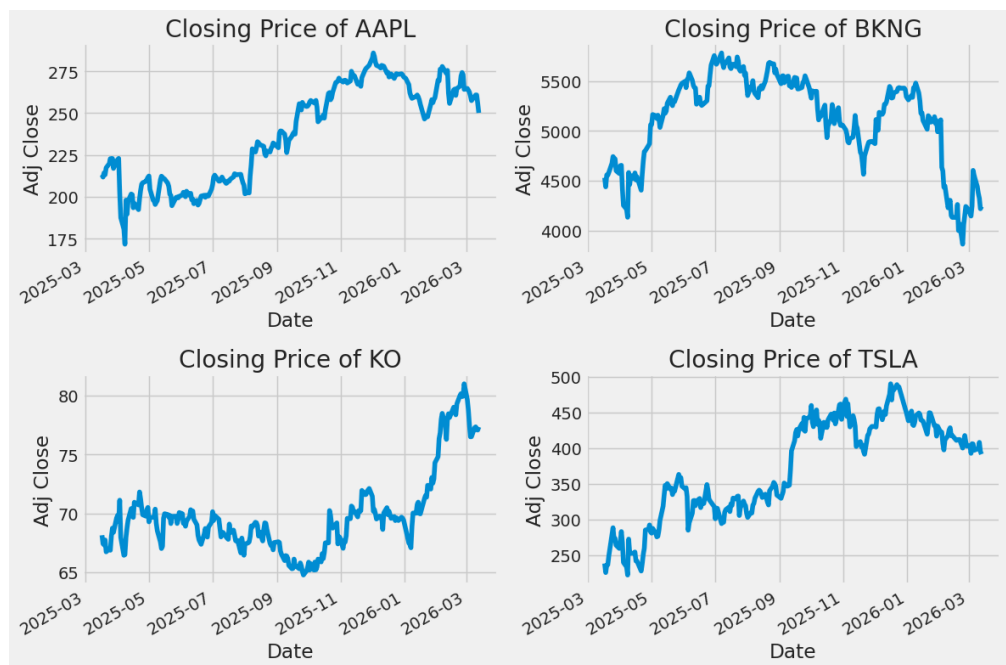
In [108]: adj_close = data.xs('Adj Close', axis=1, level=0)

# Correct figure layout
plt.figure(figsize=(12, 8))
plt.subplots_adjust(top=0.95, bottom=0.1, hspace=0.3) # top > bottom,
values between 0-1

for i, stock in enumerate(adj_close.columns, 1):
    plt.subplot(2, 2, i)
    adj_close[stock].plot()
    plt.ylabel('Adj Close')
    plt.title(f'Closing Price of {stock}')

plt.tight_layout()
plt.show()

```



Sales Volumns

```

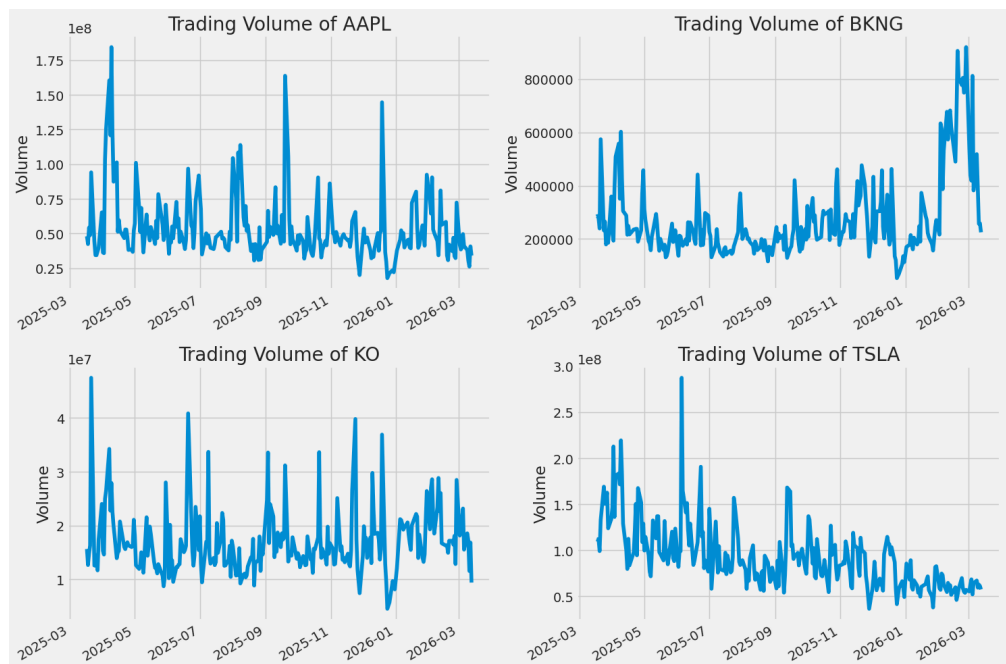
In [109]: # Extract volume data
volumes = data["Volume"]
tech_list = volumes.columns.tolist()

plt.figure(figsize=(15, 10))
plt.subplots_adjust(top=1.25, bottom=1.2)

for i, company in enumerate(tech_list, 1):
    plt.subplot(2, 2, i)
    volumes[company].plot()
    plt.ylabel('Volume')
    plt.xlabel(None)
    plt.title(f"Trading Volume of {company}")

plt.tight_layout()
plt.show()

```



2. What was the moving average of the various stocks?

```
In [110]: # Extract only the Adj Close prices
adj_close = data.xs('Adj Close', axis=1, level=0)

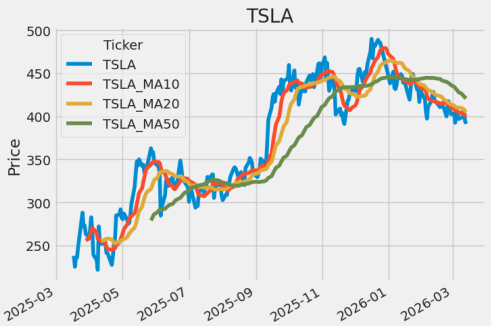
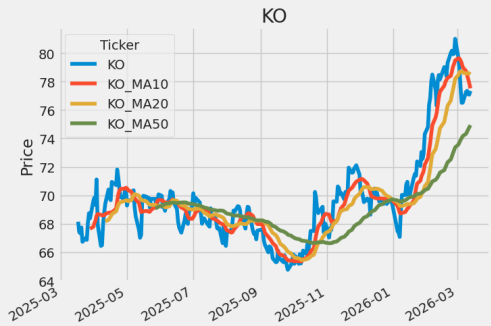
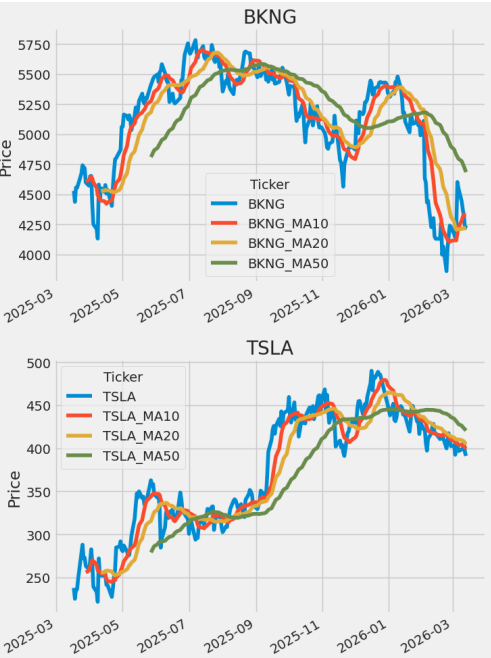
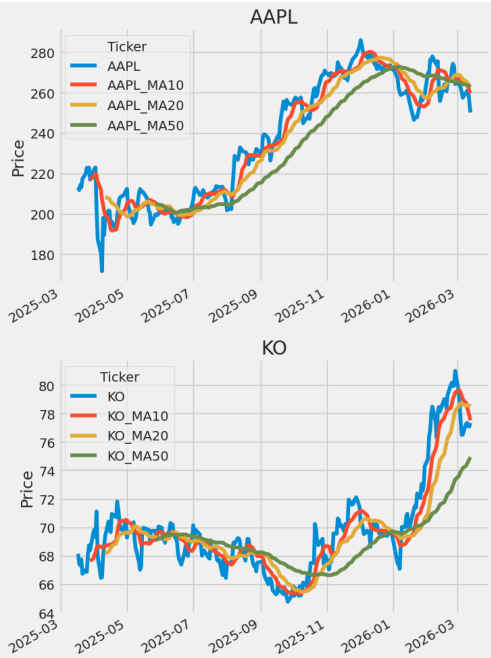
# List of companies
tech_list = adj_close.columns.tolist()
ma_days = [10, 20, 50]

# Create moving averages in a single DataFrame
ma_data = adj_close.copy()
for ma in ma_days:
    for stock in tech_list:
        ma_column = f"{stock}_MA{ma}"
        ma_data[ma_column] = adj_close[stock].rolling(ma).mean()

# Set up 2x2 subplots
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.subplots_adjust(top=0.92, hspace=0.3)

for i, stock in enumerate(tech_list):
    row = i // 2
    col = i % 2
    cols_to_plot = [stock] + [f"{stock}_MA{ma}" for ma in ma_days]
    ma_data[cols_to_plot].plot(ax=axes[row, col])
    axes[row, col].set_title(stock)
    axes[row, col].set_ylabel("Price")
    axes[row, col].set_xlabel(None)

plt.tight_layout()
plt.show()
```



3. What was the average daily return of the stock?

```
In [111]: # Extract only the Adj Close prices
adj_close = data.xs('Adj Close', axis=1, level=0)

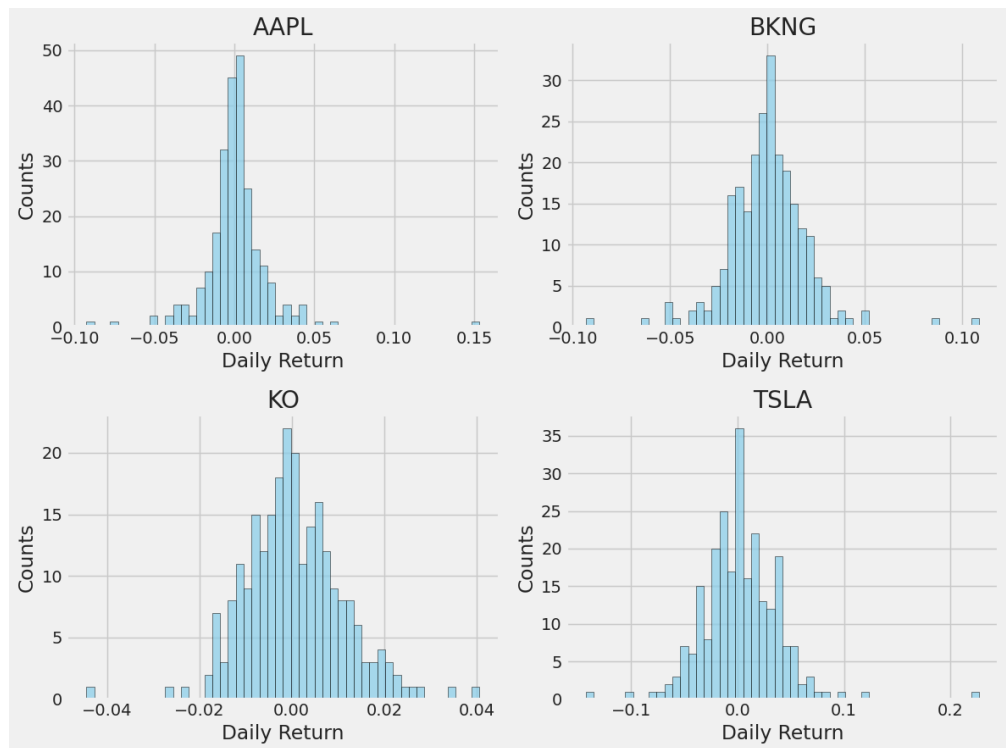
# Calculate daily returns
daily_returns = adj_close.pct_change()

# List of stocks
tech_list = adj_close.columns.tolist()

# Set up 2x2 subplot
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
fig.subplots_adjust(hspace=0.3, wspace=0.3)

# Plot histogram for each stock
for i, stock in enumerate(tech_list):
    row = i // 2
    col = i % 2
    daily_returns[stock].hist(
        bins=50,
        ax=axes[row, col],
        color='skyblue',
        edgecolor='black',
        alpha=0.7
    )
    axes[row, col].set_title(f'{stock}')
    axes[row, col].set_xlabel('Daily Return')
    axes[row, col].set_ylabel('Counts')

plt.tight_layout()
plt.show()
```



This visualization tells you how the stock behaves day-to-day rather than how it trends over time. It's a risk and volatility check.

4. What was the correlation between different stocks closing prices?

```
In [112]: # Extract only Adjusted Close prices into a simple DataFrame
closing_df = data.xs('Adj Close', axis=1, level=0)

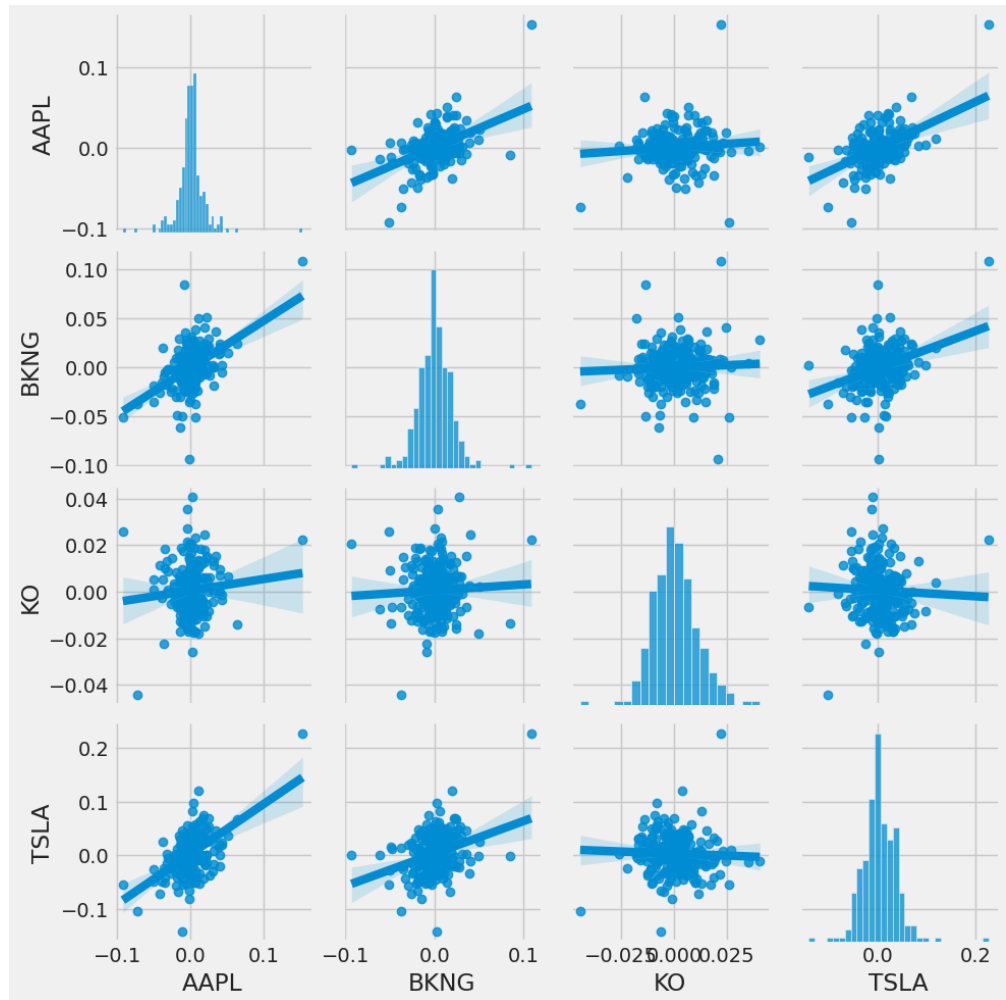
# Calculate daily returns
tech_rets = closing_df.pct_change()

# View first few rows
tech_rets.head(10)
```

Out[112]:

Ticker	AAPL	BKNG	KO	TSLA
Date				
2025-03-17	NaN	NaN	NaN	NaN
2025-03-18	-0.006122	-0.020134	-0.010553	-0.053359
2025-03-19	0.011989	0.026922	-0.001441	0.046824
2025-03-20	-0.005296	-0.000623	0.006207	0.001696
2025-03-21	0.019477	0.006152	-0.014919	0.052696
2025-03-24	0.011271	0.020025	0.004078	0.119336
2025-03-25	0.013682	0.014479	-0.002031	0.035023
2025-03-26	-0.009922	-0.002878	0.017585	-0.055806
2025-03-27	0.010473	-0.002384	0.010283	0.003933
2025-03-28	-0.026580	-0.024945	-0.005230	-0.035075

```
In [113]: sns.pairplot(tech_rets, kind='reg')  
Out[113]: <seaborn.axisgrid.PairGrid at 0x7fe57db12c00>
```



Above we can see all the relationships on daily returns between all the stocks.

* Strongest link: AAPL ↔ TSLA

→ both are large-cap "magnificent 7" style names, both very sensitive to the same macro factors (interest rates, growth expectations, tech sentiment, risk-on/risk-off days)

* Medium link: BKNG ↔ TSLA (some shared growth/consumer vibe, but TSLA is outlier volatile)

* Weakest link: Anything involving KO — it acts as the "odd one out." Coca-Cola is a classic low-beta, defensive name that doesn't swing much with tech/growth names. This makes it a natural diversifier in a portfolio heavy on AAPL/TSLA/BKNG.

```

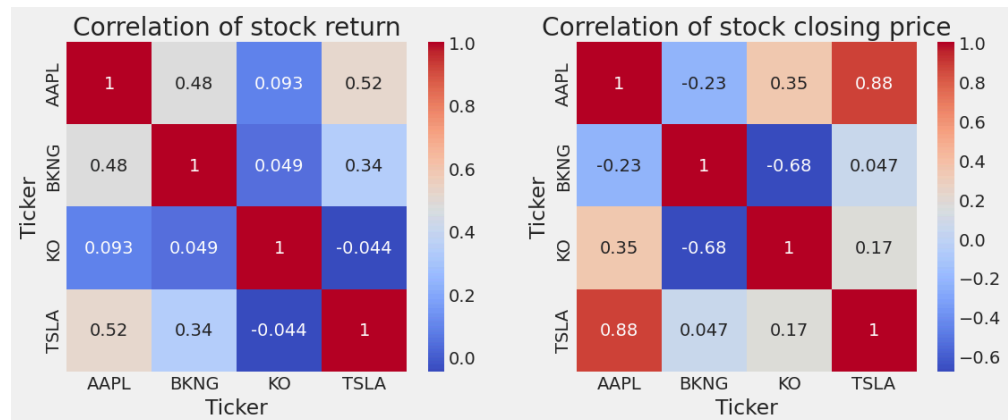
In [114]: plt.figure(figsize=(12, 10))

plt.subplot(2, 2, 1)
sns.heatmap(tech_rets.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation of stock return')

plt.subplot(2, 2, 2)
sns.heatmap(closing_df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation of stock closing price')

```

Out[114]: Text(0.5, 1.0, 'Correlation of stock closing price')



For portfolio construction, risk management, or trading pairs/hedges → focus on the left matrix (returns). AAPL & TSLA move together reasonably (0.51), KO diversifies strongly. The right matrix (prices) is mostly an artifact of long-term trends and less useful — it overstates relationships for trending stocks like AAPL/TSLA.

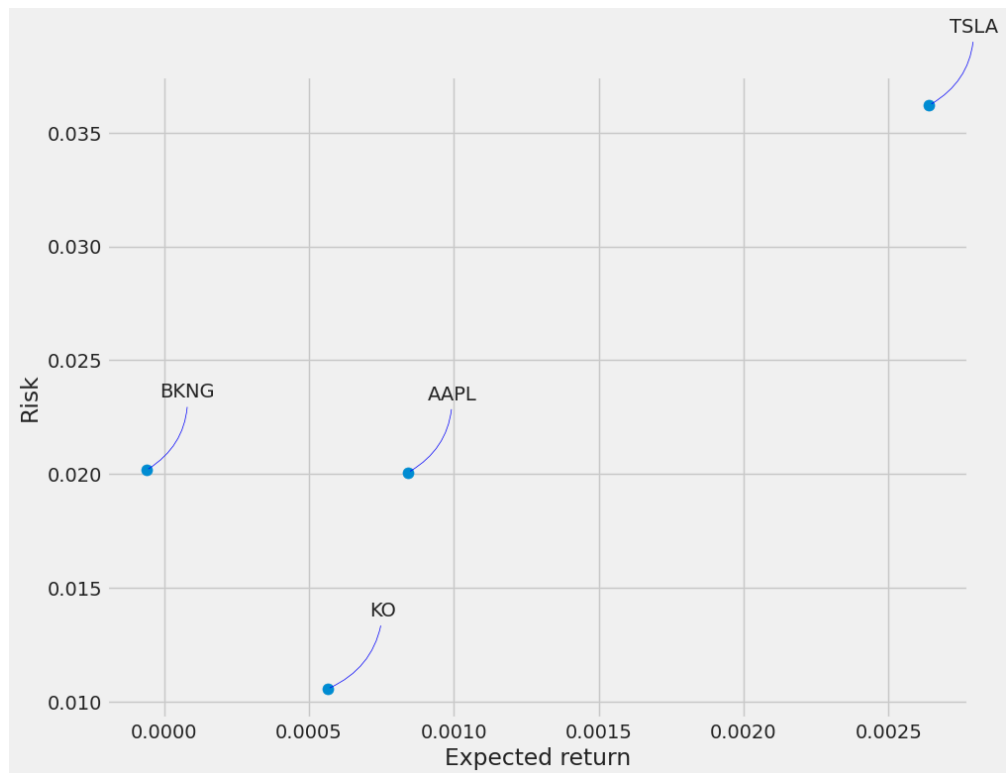
5. How much value do we put at risk by investing in a particular stock?

```
In [115]: rets = tech_rets.dropna()

area = np.pi * 20

plt.figure(figsize=(10, 8))
plt.scatter(rets.mean(), rets.std(), s=area)
plt.xlabel('Expected return')
plt.ylabel('Risk')

for label, x, y in zip(rets.columns, rets.mean(), rets.std()):
    plt.annotate(label, xy=(x, y), xytext=(50, 50), textcoords='offset
points', ha='right', va='bottom',
                arrowprops=dict(arrowstyle='-', color='blue',
connectionstyle='arc3,rad=-0.3'))
```



- * KO is the risk-minimizing choice if capital preservation is priority (lowest volatility).
- * TSLA has delivered the highest average return historically in this sample — but you paid for it with much larger daily swings.
- * AAPL sits in a "sweet spot" — good return without going to TSLA-level extremes.
- * BKNG looks relatively unattractive here: similar risk to AAPL but much lower return → underperformed on a risk-adjusted basis in this period.

6.1 Predicting the closing price stock price of APPLE inc using LSTM:

```
In [116]: # Download data
AAPL = yf.download('AAPL', start='2012-01-01', end=datetime.now(),
auto_adjust=False)
AAPL
```

[*****100%*****] 1 of 1
completed

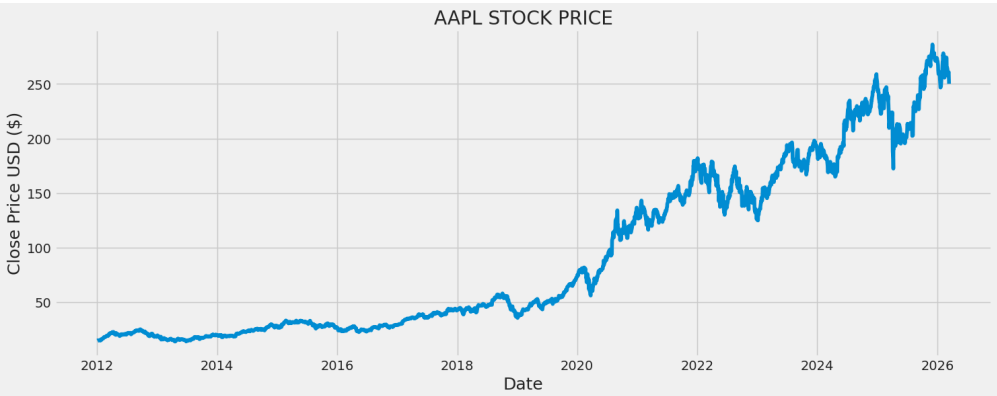
Out[116]:

Price	Adj Close	Close	High	Low	Open
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL
Date					
2012-01-03	12.321687	14.686786	14.732143	14.607143	14.621429
2012-01-04	12.387905	14.765714	14.810000	14.617143	14.642857
2012-01-05	12.525435	14.929643	14.948214	14.738214	14.819643
2012-01-06	12.656373	15.085714	15.098214	14.972143	14.991786
2012-01-09	12.636299	15.061786	15.276786	15.048214	15.196429
...	...	...	...	...	...
2026-03-09	259.880005	259.880005	261.149994	253.679993	255.690002
2026-03-10	260.829987	260.829987	262.480011	256.950012	257.649994
2026-03-11	260.809998	260.809998	262.130005	259.549988	261.089996
2026-03-12	255.759995	255.759995	258.950012	254.179993	258.660004

Price	Adj Close	Close	High	Low	Open	
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL	
Date						
2026-03-13	250.119995	250.119995	256.329987	249.520004	255.399994	

3569 rows × 6 columns

```
In [117]: plt.figure(figsize=(16,6))
plt.title('AAPL STOCK PRICE')
plt.plot(AAPL['Close'])
plt.xlabel('Date', fontsize=18)
plt.ylabel('Close Price USD ($)', fontsize=18)
plt.show()
```



```
In [118]: print(AAPL.columns)

MultiIndex([('Adj Close', 'AAPL'),
            ('Close', 'AAPL'),
            ('High', 'AAPL'),
            ('Low', 'AAPL'),
            ('Open', 'AAPL'),
            ('Volume', 'AAPL')],
           names=['Price', 'Ticker'])
```

```
In [119]: # Create a new dataframe with only the 'Close' column
df = AAPL[['Close', 'AAPL']]
log_df = np.log(df)

# Convert the dataframe to a numpy array
dataset = log_df.values
# Get the number of rows to train the model on
training_data_len = int(np.ceil( len(dataset) * 0.9 ))

training_data_len
```

Out[119]: 3213

```
In [120]: # Scale the data
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0,1))
scaled_data = scaler.fit_transform(dataset)

scaled_data
```

Out[120]: array([[0.01709427],
[0.0188682],
[0.02252247],
...,
[0.9692642],
[0.96279271],
[0.95541235]])

```

In [121]: # Create the training data set
# Create the scaled training data set
train_data = scaled_data[0:int(training_data_len), :]

# Split the data into x_train and y_train data sets
x_train = []
y_train = []

for i in range(60, len(train_data)):
    x_train.append(train_data[i-60:i, 0])
    y_train.append(train_data[i, 0])
    if i<= 61:
        print(x_train)
        print(y_train)
        print()

# Convert the x_train and y_train to numpy arrays
x_train, y_train = np.array(x_train), np.array(y_train)

# Reshape the data
x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], 1))
# x_train.shape

```

```

[array([0.01709427, 0.0188682 , 0.02252247, 0.02596448, 0.02543907,
        0.02662201, 0.02608199, 0.02517213, 0.02392879, 0.02776179,
        0.03118087, 0.03013022, 0.02431488, 0.02986703, 0.0244015 ,
        0.04444788, 0.04294023, 0.044907 , 0.04912013, 0.0516457 ,
        0.05143537, 0.05065816, 0.05395783, 0.05703236, 0.06048128,
        0.06597722, 0.07723326, 0.07740099, 0.08350221, 0.08798915,
        0.08023963, 0.08324526, 0.08318595, 0.09147246, 0.09030681,
        0.09246096, 0.09629713, 0.0984128 , 0.1044326 , 0.10875006,
        0.10998639, 0.11041773, 0.10303878, 0.10123357, 0.10150187,
        0.10847541, 0.11041166, 0.11453246, 0.12404784, 0.13633139,
        0.13406692, 0.13407256, 0.1427361 , 0.14540136, 0.14350606,
        0.14176559, 0.13994373, 0.14595799, 0.15002259, 0.15170958])]
[np.float64(0.14752470655740502)]

```

```

[array([0.01709427, 0.0188682 , 0.02252247, 0.02596448, 0.02543907,
        0.02662201, 0.02608199, 0.02517213, 0.02392879, 0.02776179,
        0.03118087, 0.03013022, 0.02431488, 0.02986703, 0.0244015 ,
        0.04444788, 0.04294023, 0.044907 , 0.04912013, 0.0516457 ,
        0.05143537, 0.05065816, 0.05395783, 0.05703236, 0.06048128,
        0.06597722, 0.07723326, 0.07740099, 0.08350221, 0.08798915,
        0.08023963, 0.08324526, 0.08318595, 0.09147246, 0.09030681,
        0.09246096, 0.09629713, 0.0984128 , 0.1044326 , 0.10875006,
        0.10998639, 0.11041773, 0.10303878, 0.10123357, 0.10150187,
        0.10847541, 0.11041166, 0.11453246, 0.12404784, 0.13633139,
        0.13406692, 0.13407256, 0.1427361 , 0.14540136, 0.14350606,
        0.14176559, 0.13994373, 0.14595799, 0.15002259, 0.15170958]),
array([0.0188682 , 0.02252247, 0.02596448, 0.02543907, 0.02662201,
        0.02608199, 0.02517213, 0.02392879, 0.02776179, 0.03118087,
        0.03013022, 0.02431488, 0.02986703, 0.0244015 , 0.04444788,
        0.04294023, 0.044907 , 0.04912013, 0.0516457 , 0.05143537,

```

```
0.05065816, 0.05395783, 0.05703236, 0.06048128, 0.06597722,  
0.07723326, 0.07740099, 0.08350221, 0.08798915, 0.08023963,  
0.08324526, 0.08318595, 0.09147246, 0.09030681, 0.09246096,  
0.09629713, 0.0984128 , 0.1044326 , 0.10875006, 0.10998639,  
0.11041773, 0.10303878, 0.10123357, 0.10150187, 0.10847541,  
0.11041166, 0.11453246, 0.12404784, 0.13633139, 0.13406692,  
0.13407256, 0.1427361 , 0.14540136, 0.14350606, 0.14176559,  
0.13994373, 0.14595799, 0.15002259, 0.15170958, 0.14752471]])  
[np.float64(0.14752470655740502), np.float64(0.14188153788531932)]
```

```

In [122]: from keras.models import Sequential
          from keras.layers import Dense, LSTM, Dropout
          from keras.callbacks import EarlyStopping

          # Building the LSTM model
          model = Sequential()
          model.add(LSTM(128, return_sequences=True, input_shape= (x_train.shape[1],
          1)))
          model.add(Dropout(0.2))
          model.add(LSTM(64, return_sequences=False))
          model.add(Dropout(0.2))
          model.add(Dense(25))
          model.add(Dense(1))

          # Compile the model
          model.compile(optimizer='adam', loss='mean_squared_error')

          # Define early stopping
          early_stop = EarlyStopping(
              monitor='val_loss',    # Monitor validation loss
              patience=10,           # Stop if no improvement for 10 epochs
              restore_best_weights=True # Restore weights of best epoch
          )

          # Train the model with validation split and early stopping
          history = model.fit(
              x_train, y_train,
              batch_size=32,
              epochs=20,
              validation_split=0.1,    # Use 10% of training data as validation
              callbacks=[early_stop]
          )

```

```

Epoch 1/20
1/1 [1m89/89][0m 32m-----[0m 37m[0m 1m17s[0m
124ms/step - loss: 0.0080 - val_loss: 0.0016
Epoch 2/20
1/1 [1m89/89][0m 32m-----[0m 37m[0m 1m16s[0m
76ms/step - loss: 0.0015 - val_loss: 2.1395e-04
Epoch 3/20
1/1 [1m89/89][0m 32m-----[0m 37m[0m 1m7s[0m
79ms/step - loss: 0.0012 - val_loss: 1.2896e-04
Epoch 4/20
1/1 [1m89/89][0m 32m-----[0m 37m[0m 1m11s[0m
125ms/step - loss: 0.0011 - val_loss: 5.5985e-04
Epoch 5/20
1/1 [1m89/89][0m 32m-----[0m 37m[0m 1m16s[0m
73ms/step - loss: 9.7241e-04 - val_loss: 1.0631e-04
Epoch 6/20
1/1 [1m89/89][0m 32m-----[0m 37m[0m 1m7s[0m
81ms/step - loss: 8.6472e-04 - val_loss: 4.8967e-04
Epoch 7/20
1/1 [1m89/89][0m 32m-----[0m 37m[0m 1m7s[0m

```

75ms/step - loss: 7.1610e-04 - val_loss: 0.0011
Epoch 8/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
81ms/step - loss: 6.9367e-04 - val_loss: 1.4396e-04
Epoch 9/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
83ms/step - loss: 6.0847e-04 - val_loss: 6.9054e-04
Epoch 10/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
74ms/step - loss: 5.7915e-04 - val_loss: 9.5487e-05
Epoch 11/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m8s][0m
89ms/step - loss: 5.9741e-04 - val_loss: 9.1645e-05
Epoch 12/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
77ms/step - loss: 6.3405e-04 - val_loss: 2.8471e-04
Epoch 13/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
83ms/step - loss: 5.0234e-04 - val_loss: 5.3419e-04
Epoch 14/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m10s][0m
78ms/step - loss: 5.0099e-04 - val_loss: 1.0386e-04
Epoch 15/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m8s][0m
87ms/step - loss: 5.1726e-04 - val_loss: 1.9853e-04
Epoch 16/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
73ms/step - loss: 5.1357e-04 - val_loss: 2.3137e-04
Epoch 17/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m10s][0m
74ms/step - loss: 5.0322e-04 - val_loss: 2.3655e-04
Epoch 18/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m11s][0m
77ms/step - loss: 4.4592e-04 - val_loss: 1.2055e-04
Epoch 19/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
79ms/step - loss: 5.2756e-04 - val_loss: 1.0392e-04
Epoch 20/20
[1m89/89][0m][32m]-----[0m][37m][0m][1m7s][0m
75ms/step - loss: 4.5425e-04 - val_loss: 1.8582e-04

In
[123]:

```
from sklearn.metrics import mean_squared_error

# Create the testing data set
test_data = scaled_data[training_data_len - 60: , :]

# Create the data sets x_test and y_test
x_test = []
y_test = dataset[training_data_len:, :]

for i in range(60, len(test_data)):
    x_test.append(test_data[i-60:i, 0])

# Convert the data to a numpy array
x_test = np.array(x_test)

# Reshape the data
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1 ))

# Get the models predicted price values
predictions = model.predict(x_test)
# Reverse scaling
predictions = scaler.inverse_transform(predictions)
# Convert log prices back to real prices
predictions = np.exp(predictions)
y_test_actual = np.exp(y_test)

mse_lstm = mean_squared_error(y_test_actual.flatten(),
                              predictions.flatten())
rmse_lstm = np.sqrt(mse_lstm)

print("LSTM RMSE:", rmse_lstm)
```

12/12 0m 32m ————— 0m 37m 0m 1s 0m
36ms/step
LSTM RMSE: 8.292641984882845

In
[124]:

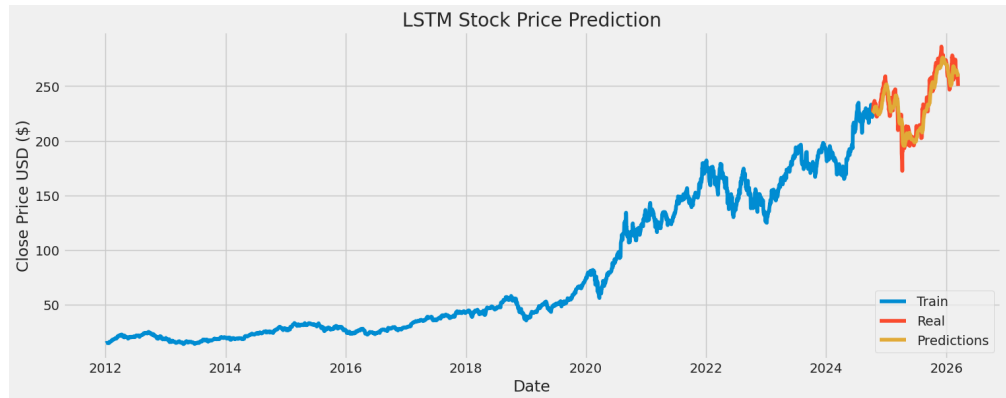
```
# Creating a dataframe with only Close prices
data_plot = df.copy()
data_plot['Close'] = df['Close']
train = data_plot[:training_data_len]
valid = data_plot[training_data_len:].copy()
valid['Predictions'] = predictions.flatten()
```

```
In [125]: plt.figure(figsize=(16,6))

plt.title('LSTM Stock Price Prediction')
plt.xlabel('Date')
plt.ylabel('Close Price USD ($)')

plt.plot(train['Close'], label='Train')
plt.plot(valid['Close'], label='Real')
plt.plot(valid['Predictions'], label='Predictions')

plt.legend(loc='lower right')
plt.show()
```



In
[126]:

valid

Out[126]:

Price	Close	Predictions
Ticker	AAPL	
Date		
2024-10-10	229.039993	225.232513
2024-10-11	227.550003	225.401093
2024-10-14	231.300003	225.562820
2024-10-15	233.850006	225.964890
2024-10-16	231.779999	226.661377
...	...	...
2026-03-09	259.880005	261.846039
2026-03-10	260.829987	260.840729
2026-03-11	260.809998	260.017303
2026-03-12	255.759995	259.367310
2026-03-13	250.119995	258.521729

356 rows × 2 columns

6.2 Predicting the closing price stock price of APPLE inc using ARIMA:

In
[127]:

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
```

```

In      #Taking the AAPL df created in LSTM
[128]: log_df = np.log(df)
        log_prices = np.log(df[('Close', 'AAPL')]) #Converting df to series

        train_size= int(np.ceil( len(log_prices) * 0.9 ))

        train_log = log_prices.iloc[:train_size]
        test_log  = log_prices.iloc[train_size:]

        # Dates for plotting
        test_dates = log_prices.index[train_size:]

        print(len(train_log))
        print(len(test_log))

```

```

3213
356

```

```

In      # ADF test
[129]: result = adfuller(train_log.dropna())
        print('ADF Statistic:', result[0])
        print('p-value:', result[1])
        # If p > 0.05 → not stationary → you'll need differencing (d≥1 in ARIMA)

```

```

ADF Statistic: -0.26819525623422946
p-value: 0.9298896085435633

```

Since the pvalue is greater, the null hypothesis of "Non Stationarity" cannot be rejected, therefore differencing is necessary.

```

In      # Differencing by 1 time
[130]: log_diff_1 = train_log.diff().dropna()

```

```

In      from statsmodels.tsa.stattools import adfuller
[131]: result = adfuller(log_diff_1)

        print('ADF Statistic:', result[0])
        print('p-value:', result[1])
        print('Critical Values:')
        for key, value in result[4].items():
            print(f'\t{key}: {value:.4f}')

```

```

ADF Statistic: -18.25406990823856
p-value: 2.3392721086777846e-30
Critical Values:
    1%: -3.4324
    5%: -2.8624
   10%: -2.5673

```

ADF statistic is more negative than the critical values → Therefore now the process is stationary

```

In [132]: !pip install pmdarima
          from pmdarima import auto_arima
          from statsmodels.tsa.arima.model import ARIMA

          auto_model = auto_arima(
              train_log,                                # ← original log prices (not
differenced)
              start_p=1, start_q=1,
              max_p=5, max_q=5,
              d=None,                                   # ← let auto_arima find the right
differencing
              seasonal=False,
              trace=True,
              error_action='ignore',
              suppress_warnings=True,
              stepwise=True,
              maxiter=50
          )

          print("Best order found:", auto_model.order)
          print(auto_model.summary())

          arima_fit = ARIMA(train_log, order=auto_model.order).fit()

```

```

Requirement already satisfied: pmdarima in
/usr/local/lib/python3.12/dist-packages (2.1.1)
Requirement already satisfied: joblib>=0.11 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (1.5.3)
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (3.0.12)
Requirement already satisfied: numpy>=1.21.6 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (2.0.2)
Requirement already satisfied: pandas>=0.19 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (2.2.2)
Requirement already satisfied: scikit-learn>=0.22 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (1.6.1)
Requirement already satisfied: scipy>=1.13.0 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (1.16.3)
Requirement already satisfied: statsmodels>=0.14.5 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (0.14.6)
Requirement already satisfied: urllib3 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (2.5.0)
Requirement already satisfied: setuptools!=50.0.0,>=42 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (75.2.0)
Requirement already satisfied: packaging>=17.1 in
/usr/local/lib/python3.12/dist-packages (from pmdarima) (26.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas>=0.19->pmdarima)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas>=0.19->pmdarima)
(2025.2)
Requirement already satisfied: tzdata>=2022.7 in

```

```

/usr/local/lib/python3.12/dist-packages (from pandas>=0.19->pmdarima)
(2025.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.22-
>pmdarima) (3.6.0)
Requirement already satisfied: patsy>=0.5.6 in
/usr/local/lib/python3.12/dist-packages (from statsmodels>=0.14.5-
>pmdarima) (1.0.2)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas>=0.19->pmdarima) (1.17.0)

```

Performing stepwise search to minimize aic

```

ARIMA(1,1,1)(0,0,0)[0] intercept : AIC=-16761.092, Time=0.71 sec
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=-16757.298, Time=0.29 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=-16762.429, Time=0.20 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AIC=-16762.573, Time=0.35 sec
ARIMA(0,1,0)(0,0,0)[0]           : AIC=-16751.887, Time=0.13 sec
ARIMA(0,1,2)(0,0,0)[0] intercept : AIC=-16760.869, Time=0.39 sec
ARIMA(1,1,2)(0,0,0)[0] intercept : AIC=-16759.111, Time=1.46 sec
ARIMA(0,1,1)(0,0,0)[0]           : AIC=-16756.407, Time=0.20 sec

```

Best model: ARIMA(0,1,1)(0,0,0)[0] intercept

Total fit time: 3.729 seconds

Best order found: (0, 1, 1)

SARIMAX Results

```

=====
=====
Dep. Variable:          y      No. Observations:
3213
Model:                SARIMAX(0, 1, 1)      Log Likelihood
8384.287
Date:                Sun, 15 Mar 2026      AIC
-16762.573
Time:                06:30:36      BIC
-16744.350
Sample:                0      HQIC
-16756.041
                        - 3213
Covariance Type:          opg
=====
=====

```

	coef	std err	z	P> z	[0.025
intercept	0.0008	0.000	2.804	0.005	0.000
ma.L1	-0.0479	0.012	-4.048	0.000	-0.071
sigma2	0.0003	4.32e-06	73.217	0.000	0.000

```

=====
=====

```

```

Ljung-Box (L1) (Q):                0.00   Jarque-Bera (JB):
4169.39
Prob(Q):                           0.99   Prob(JB):
0.00
Heteroskedasticity (H):             1.14   Skew:
-0.28
Prob(H) (two-sided):               0.03   Kurtosis:
8.55
=====
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients
(complex-step).

```

In
[133]:

```

from statsmodels.tsa.arima.model import ARIMA

order=auto_model.order

history = train_log.values.tolist()
arima_preds_log = []

for t in range(len(test_log)):

    model = ARIMA(history, order=auto_model.order)
    model_fit = model.fit()

    forecast = model_fit.forecast()[0]
    arima_preds_log.append(forecast)

    history.append(test_log.iloc[t])

arima_predictions = np.exp(np.array(arima_preds_log))

actual_prices = np.exp(test_log.values)

mse_arima = mean_squared_error(actual_prices, arima_predictions)
rmse_arima = np.sqrt(mse_arima)
print("ARIMA RMSE:", rmse_arima)

```

ARIMA RMSE: 4.0939588070914095

```

In [134]: train_prices = np.exp(train_log)      # back to original scale
          actual_test_prices = np.exp(test_log) # back to original scale

          train_plot = pd.Series(train_prices, index=train_log.index)
          test_actual_plot = pd.Series(actual_test_prices, index=test_log.index)
          test_pred_plot = pd.Series(arima_predictions, index=test_log.index)

          # — Plot —————
          plt.figure(figsize=(16, 6))

          plt.title('ARIMA Stock Price Prediction (AAPL)')
          plt.xlabel('Date')
          plt.ylabel('Close Price USD ($)')

          # Training period
          plt.plot(train_plot.index, train_plot,
                   label='Train', color='blue', linewidth=1.0)

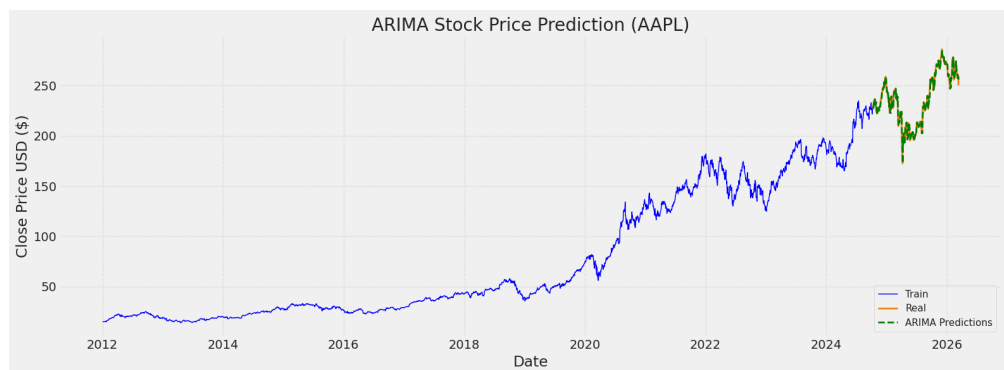
          # Test period – actual
          plt.plot(test_actual_plot.index, test_actual_plot,
                   label='Real', color='#f57c00', linewidth=1.8)

          # Test period – ARIMA prediction
          plt.plot(test_pred_plot.index, test_pred_plot,
                   label='ARIMA Predictions', color='green', linewidth=2.0,
                   linestyle='--')

          plt.legend(loc='lower right', fontsize=11)
          plt.grid(True, alpha=0.3, linestyle='--')

          plt.tight_layout()
          plt.show()

```



Comparative Analysis of Stock Price Forecasting: LSTM vs. ARIMA

In this project, we successfully implemented a financial data pipeline and compared two forecasting models for Apple's stock price.

After training and validating both models on Apple's stock data, the results were as follows:

- * ARIMA RMSE: ~4.09

- * LSTM RMSE: ~8.29

Key Findings:

In this specific study, the ARIMA model significantly outperformed the LSTM model. While LSTM is a powerful tool for complex patterns, the superior performance of ARIMA suggests that for this specific timeframe, the stock's movement was more closely tied to its immediate historical trends and mean-reverting properties.

Lessons Learned:

- * Model Simplicity: This result highlights a common phenomenon in quantitative finance—sometimes "simpler" statistical models are more robust against market noise than complex neural networks.

- * Forecasting Reality: Stock prices are often "random walks," and the ARIMA model's focus on differencing and moving averages provided a more accurate "fit" for the validation data.

This project successfully demonstrates a full Data Science workflow: from data acquisition and risk modeling to a rigorous comparison of Machine Learning vs. Statistical Inference.